

A sound pure type system with ranks as sorts and Type-in-Type

Pavel Sokolov

October 10, 2026

Abstract

We present a dependently typed pure type system which contains a circular axiom $s : s$ for a certain sort s and is nonetheless strongly normalizing. Girard’s paradox is avoided because our sorts are not closed under a Π -former: function types are placed in sorts according to their *rank*. We prove strong normalization of a system and provide a bare-bones set-theoretical model for it. A normalization proof is surprisingly simple (modulo general metatheoretical properties of pure type systems) and has been fully formalized in vanilla Rocq (formerly Coq) without any additional axioms.

1 Introduction

Pure type systems (PTSs) are a family of minimalistic typed λ -calculi including grammar formers for variables $x \in \mathbf{Var}$, abstraction $\lambda(x : T).t$, application $t t'$, types of dependent functions $\Pi(x : T).U$ (where U might depend on x) and *sorts* $s \in \mathcal{S}$ which act like “types of types”. In contrast with many other type systems, there is no syntactical distinction between terms and types — they are defined by a single grammar; an expression can act both as a term and as a type depending on a side where it stands next to the colon in the typing judgement $\Gamma \vdash _ : _$. A single PTS is determined by two relations:

- *axiom* relation $\mathbf{Ax} \subseteq \mathcal{S} \times \mathcal{S}$ determines “to which sort(s) a given sort belongs as an element”; instead of $(s, s') \in \mathbf{Ax}$ it is customary to write simply $s : s'$.
- *rule* relation $\mathbf{Rule} \subseteq \mathcal{S}^2 \times \mathcal{S}$ places types of functions $\Pi(x : T).U$ into sorts accounting for the sorts where T and U are situated.

Systems from Barendregt’s lambda cube fit naturally into the framework: for example, simply typed lambda calculus (STLC) can be viewed as a pure type system with a single sort $*$, no axioms and a single rule $(*, *, *)$. In general, however, a pure type system is dependently typed since Π allows the codomain

$$\begin{array}{c}
\text{Ax} \\
\frac{}{\Gamma \vdash s : s'} (s, s') \in \text{Ax}
\end{array}
\quad
\begin{array}{c}
\text{Rule} \\
\frac{\Gamma \vdash T : s_T \quad \Gamma, x : T \vdash U : s_U}{\Gamma \vdash \Pi(x : T).U : s} (s_T, s_U, s) \in \text{Rule}
\end{array}$$

$$\begin{array}{c}
\text{Var} \\
\frac{}{\Gamma \vdash x : T} x : T \in \Gamma
\end{array}
\quad
\begin{array}{c}
\text{Abs} \\
\frac{\Gamma \vdash T : s_T \quad \Gamma, x : T \vdash u : U}{\Gamma \vdash \lambda(x : T).u : \Pi(x : T).U}
\end{array}$$

$$\begin{array}{c}
\text{App} \\
\frac{\Gamma \vdash u : \Pi(x : T).U \quad \Gamma \vdash t : T}{\Gamma \vdash ut : U[t/x]}
\end{array}
\quad
\begin{array}{c}
\text{Conv} \\
\frac{\Gamma \vdash t : T \quad \Gamma \vdash U : s \quad T =_{\beta} U}{\Gamma \vdash t : U}
\end{array}$$

Figure 1: Typing rules of pure type systems

type to depend on the value of an argument — Coquand’s Calculus of Constructions (CoC) sits comfortably at the top of lambda cube and is modeled as a pure type system with two sorts $\{*, \square\}$, a single axiom $* : \square$ and four rules corresponding to simple functions $(*, *, *)$, polymorphic functions $(\square, *, *)$, type operators $(\square, \square, \square)$ and dependent types $(*, \square, \square)$.

As is known, both STLC and CoC are strongly normalizing and sound with respect to their set-theoretical semantics. A classical result shows that it is not true for arbitrary PTS.

Theorem (Girard’72). *The following PTS called System U is not normalizing:*

$$\begin{aligned}
\mathcal{S} &= \{*, \square, \triangle\}, & \text{Ax} &= \{(*, \square), (\square, \triangle)\}, \\
\text{Rule} &= \{(*, *, *), (\square, *, *), (\triangle, *, *), (\square, \square, \square), (\triangle, \square, \square)\}
\end{aligned}$$

*Futhermore, the offending term has type $\Pi(P : *).P$.*

This result is dubbed “Girard’s paradox” since it renders System U inconsistent with respect to its intended logical semantics: inhabitedness of $\Pi(P : *).P$ would mean that every proposition is provable. Thus any system where System U embeds is inconsistent, too; in particular, an original version of Martin-Löf’s Intuitionistic Type Theory embeds the following PTS known as “Type-in-Type” which is inconsistent due to Girard’s paradox:

$$\mathcal{S} = \{*\}, \quad \text{Ax} = \{* : *\}, \quad \text{Rule} = \{(*, *, *)\}.$$

For a pure type system with usual set-theoretical semantics, it is natural to expect that a sort of *small types* $*$ is closed under Π , thus we cannot forbid the rule $(*, *, *)$; then, a circular axiom $* : *$ has to go. Long story short, it motivates an introduction of a *universe hierarchy* $\mathcal{S} = \{\mathcal{U}_k\}_{k \in \mathbb{N}}$ with axioms $\mathcal{U}_k : \mathcal{U}_{k+1}$ and rules $(\mathcal{U}_k, \mathcal{U}_l, \mathcal{U}_{k \vee l})$ for each k, l ; when interpreted in sets, $\llbracket \mathcal{U}_k \rrbracket$ is taken to be the k -th inaccessible cardinal.

Observing this, we took interest in the following two questions:

- If we were to drop the rule $(*, *, *)$ and keep the axiom $* : *$ instead, can we find a nontrivial consistent pure type system?
- Is there a consistent pure type system with total relations **Ax** and **Rule** whose natural set-theoretical semantics do not involve assumption of inaccessible cardinals?

Turns out that, for both questions, answer is “yes”. Furthermore, both yes-s are witnessed by a single PTS, which we dub **PTS- rk** :

$$\mathcal{S} = \{rk_r\}_{r \in \mathbb{N}} \quad \mathbf{Ax} = \{(rk_r, rk_0)\}_{r \in \mathbb{N}} \quad \mathbf{Rule} = \{(rk_r, rk_s, rk_{(r+1) \vee s})\}_{r, s \in \mathbb{N}}$$

(Naming “**rk**” comes from *rank*, since rules compute precisely the rank of functions, which is the maximal nesting of Π in the domain type.)

Indeed, there is a circular axiom $rk_0 : rk_0$, and System **U** is not embeddable since rk_r is not closed under Π for any r . Also, as we will see in Section 3, our **PTS- rk** is strongly normalizing and has a set-theoretical model in ZFC despite an intuition that $rk_r : rk_0$ might “feel wrong”. Modulo general metatheory of pure type systems laid out in Section 2, a normalization proof turned out to be suspiciously simple, even more so in comparison with normalization proofs of other dependently typed systems. To dispel our own doubts, we formalized a proof of normalization in Rocq (formerly Coq) without using any additional axioms, yielding a fully constructive proof; our journey in formalization is summarised in Section 4. We provide a closing summary and some pointers on possible extensions of the system in Section 5.

2 Metatheory of pure type systems

For ease of presentation of our results, in Sections 2-3 we take a Barendregt convention which assumes that all bound variables are distinct from free variables and from each other; furthermore, α -equivalence (renaming of bound variables) is assumed as a trivial equality on terms. Nevertheless, our formalization takes variable treatment seriously; see Section 4 for more details.

2.1 PTS as a rewriting system

In Fig. 1, a rule of *conversion* (**CONV**) mentions *definitional equality* of types $T =_\beta U$. It is defined as a reflexive, symmetric, transitive closure of a one-step reduction.

Definition 1 (One-step reduction). Relation of a one-step reduction $t \rightarrow_\beta t'$ is

the smallest relation satisfying the following rules:

$$\begin{array}{c}
\frac{T \rightarrow_\beta T'}{\Pi(x : T).U \rightarrow_\beta \Pi(x : T').U} \quad \frac{U \rightarrow_\beta U'}{\Pi(x : T).U \rightarrow_\beta \Pi(x : T).U'} \\
\\
\frac{T \rightarrow_\beta T'}{\lambda(x : T).t \rightarrow_\beta \lambda(x : T').t} \quad \frac{t \rightarrow_\beta t'}{\lambda(x : T).t \rightarrow_\beta \lambda(x : T).t'} \quad \frac{t \rightarrow_\beta t'}{tu \rightarrow_\beta t'u} \\
\\
\frac{u \rightarrow_\beta u'}{tu \rightarrow_\beta tu'} \quad \text{BETA} \quad (\lambda(x : T).t)u \rightarrow_\beta t[u/x]
\end{array}$$

In essence, actual reduction happens only in rule BETA; other rules simply state the we can freely reduce any subterm.

The following result will be greatly useful in our development. It was proven many times over for different similar systems; we briefly revisit the proof since previous presentations usually did not allow reductions in the type of argument T in $\lambda(x : T).t$.

Theorem 1 (Characterization of $(=_\beta)$). *In arbitrary PTS,*

$$t =_\beta u \quad \Leftrightarrow \quad \exists v. t \rightarrow_\beta^* v \wedge u \rightarrow_\beta^* v,$$

where $(\rightarrow_\beta^*)$ is reflexive transitive closure of $(\rightarrow_\beta)$.

Proof. Note that it is enough to prove that $(\rightarrow_\beta^*)$ has a diamond property — for each t_0, t_1, t_2 such that $t_0 \rightarrow_\beta^* t_1$ and $t_0 \rightarrow_\beta^* t_2$ there exists a t' such that $t_1 \rightarrow_\beta^* t'$ and $t_2 \rightarrow_\beta^* t'$:

$$\begin{array}{ccc}
t_0 & \longrightarrow & t_1 \\
\downarrow & & \downarrow \\
t_2 & \dashrightarrow & t'
\end{array}$$

Then, any “mountains” in the path $t =_\beta u$ can be switched into “valleys”.

Now, diamond property is proven by standard technique of parallel reduction which is defined as the smallest relation satisfying:

$$\begin{array}{c}
t \Rightarrow t \quad \frac{T \Rightarrow T' \quad U \Rightarrow U'}{\Pi(x : T).U \Rightarrow \Pi(x : T').U'} \quad \frac{T \Rightarrow T' \quad t \Rightarrow t'}{\lambda(x : T).t \Rightarrow \lambda(x : T').t'} \\
\\
\frac{t \Rightarrow t' \quad u \Rightarrow u'}{tu \Rightarrow t'u'} \quad \frac{t \Rightarrow t' \quad u \Rightarrow u'}{(\lambda(x : T).t)u \Rightarrow t'[u'/x]}
\end{array}$$

Intuitively, while $(\rightarrow_\beta)$ is a single β -reduction somewhere in the term and $(\rightarrow_\beta)^*$ is a sequence of reductions, $(\Rightarrow)$ is several simultaneous reductions immediately available in the term. Notice, then, that $(\rightarrow_\beta) \subseteq (\Rightarrow) \subseteq (\rightarrow_\beta^*)$; by monotonicity of reflexive transitive closure, $(\rightarrow_\beta^*) \subseteq (\Rightarrow^*) \subseteq (\rightarrow_\beta^*)^* = (\rightarrow_\beta^*)$, so $(\Rightarrow^*) = (\rightarrow_\beta^*)$. Thus it is enough to prove diamond property of $(\Rightarrow^*)$. Diamond property is preserved under reflexive transitive closure, so it is enough to prove diamond property of $(\Rightarrow)$, which holds by tedious case analysis. $\square$

Setting aside practical applications of this theorem (to algorithmically check equivalence of terms, we can now search for common reduction result), it will prove useful already in our proof development. For example, the following “equality inversion” lemmas are direct corollaries:

Corollary 1 (Equality inversion). *If $s =_\beta s'$, then $s = s' \in \mathcal{S}$; if $\Pi(x : T).U =_\beta \Pi(x : T').U'$, then $T =_\beta T'$ and $U =_\beta U'$.*

They will become necessary in proving subject reduction, or type preservation. But first, we will show some structural properties of a typing judgement given in Fig. 1.

2.2 Properties of a typing judgement

Again, they surely have been proven before for PTS, but we slightly differ in the formulation of typing rules in important places so it’s better to recheck that these hold.

Lemma 1 (Weakening). *The following rule is admissible:*

$$\frac{\Gamma, \Delta \vdash t : T}{\Gamma, x : U, \Delta \vdash t : T}.$$

Proof. By structural induction on the typing derivation. □

Lemma 2 (Substitution). *The following rule is admissible:*

$$\frac{\Gamma, x : U, \Delta \vdash t : T \quad \Gamma \vdash u : U}{\Gamma, \Delta[u/x] \vdash t[u/x] : T[u/x]}.$$

Proof. By structural induction on the typing derivation for t . For variables in Δ , it is necessary that both Δ and T get substitutions of u for x so VAR is still applicable; for places in t where x used to be we need to apply weakening lemma to the typing derivation for u exactly $|\Delta|$ times. In the case of CONV it is important that definitional equality of types is preserved by substitution (thanks to characterization of $(=_\beta)$). □

In addition, a family of so-called *inversion lemmas* stating that type of well-typed term is largely determined by shape of a term are also proven by straightforward induction on typing derivation (each one is proven separately, no mutual induction involved):

Lemma 3 (Inversion lemmas).

$$\Gamma \vdash s : T \Rightarrow \exists s'. (s, s') \in \text{Ax} \wedge T =_{\beta} s' \quad (1)$$

$$\Gamma \vdash \Pi(x : T).U : V \Rightarrow \exists s_T, s_U, s. \begin{cases} (s_T, s_U, s) \in \text{Rule} \\ \Gamma \vdash T : s_T \\ \Gamma, x : T \vdash U : s_U \\ V =_{\beta} s \end{cases} \quad (2)$$

$$\Gamma, x : T, \Delta \vdash x : U \Rightarrow T =_{\beta} U \quad (3)$$

$$\Gamma \vdash \lambda(x : T).t : V \Rightarrow \exists s, U. \begin{cases} \Gamma \vdash T : s \\ \Gamma, x : T \vdash t : U \\ V =_{\beta} \Pi(x : T).U \end{cases} \quad (4)$$

$$\Gamma \vdash tu : V \Rightarrow \exists U, T. \begin{cases} \Gamma \vdash t : \Pi(x : U).T \\ \Gamma \vdash u : U \\ V =_{\beta} T[u/x] \end{cases} \quad (5)$$

To progress further, we need to make certain assumptions on the PTS we have in question.

Definition 2 (Functional PTS). A pure type system is called *functional* if $\text{Ax} \subseteq \mathcal{S} \times \mathcal{S}$ and $\text{Rule} \subseteq \mathcal{S}^2 \times \mathcal{S}$ are functional relations, that is,

$$(s : s') \wedge (s : s'') \Rightarrow s' = s'' \quad \text{and} \quad (s_1, s_2, s) \in \text{Rule} \wedge (s_1, s_2, s') \in \text{Rule} \Rightarrow s = s'$$

Definition 3 (Total PTS). A pure type system is called *total* if $\text{Ax} \subseteq \mathcal{S} \times \mathcal{S}$ and $\text{Rule} \subseteq \mathcal{S}^2 \times \mathcal{S}$ are total relations, that is,

$$\forall s. \exists s'. (s, s') \in \text{Ax} \quad \text{and} \quad \forall s_1, s_2. \exists s. (s_1, s_2, s) \in \text{Rule}$$

Theorem 2 (Uniqueness of typing). *In a functional PTS, the following rule is admissible:*

$$\frac{\Gamma \vdash t : T \quad \Gamma \vdash t : T'}{T =_{\beta} T'}.$$

Proof. By structural induction on t and inversion of given typing derivations.

- $t = s \in \mathcal{S}$: by functionality of Ax .
- $t = \Pi(x : T).U$: inversion of typing gives a pair of (possibly different) sorts for T and another pair for U . By induction we get that ranks inside pairs are definitionally equal, which by equality inversion gives strictly equal sorts. Then what is needed holds by functionality of Rule .
- Case for $t = x \in \text{Var}$ holds trivially.
- $t = (\lambda(x : T).u)$: by induction hypotheses and by congruence of $(=_{\beta})$.
- $t = uv$: by typing inversion and by induction hypothesis for u we get $u : \Pi(x : T_1).U_1$ and $\Pi(x : T_1).U_1 =_{\beta} \Pi(x : T_2).U_2$. We need $U_1[v/x] =_{\beta} U_2[v/x]$ which we get from equality inversion and preservation of def.eq. under substitution.

□

In our proof of normalization, we're going to rely on yet another property: if there is a derivation for $t : T$, then T is itself typeable with some sort. However, it doesn't hold in our current formulation of typing rules: in $\Gamma \vdash x : T$, nothing prevents a context Γ to contain an entry $(x : T')$ where T' is not typeable. Thus we need an additional assumption.

Definition 4 (Context well-formedness). Context Γ is *well-formed* (written $\vdash \Gamma$) if every type in it is typed by some sort. Formally, set of well-formed contexts is the least set closed by the following rules:

$$\frac{}{\vdash \cdot} \quad \text{and} \quad \frac{\vdash \Gamma \quad \Gamma \vdash T : s}{\vdash \Gamma, x : T}$$

(where $\cdot$ denotes an empty context.)

Usually, when presenting a dependently typed system, context well-formedness is added as a premise to typing rules which would otherwise be premise-free (such as AX and VAR in a PTS). Here we instead provide it as an additional assumption only when necessary; in addition to leading to simpler metatheory (we avoid an otherwise mutually inductive definition of $(\vdash _)$ and $(_ \vdash _ : _)$), this is closer to how decision procedures for typing judgements are implemented.

Theorem 3 (Typing yields correct type). *In a total PTS, if $\Gamma \vdash t : T$ and $\vdash \Gamma$, then there exists $s \in \mathcal{S}$ such that $\Gamma \vdash T : s$.*

Proof. By structural induction on typing derivation.

- AX, RULE: by totality of Ax.
- VAR: follows from $\vdash \Gamma$.
- ABS: by totality of Rule.
- APP: $\Gamma \vdash tu : T[u/x]$. From induction hypothesis for t we know that $\Gamma \vdash t : \Pi(x : U).T : s$ for some s , then by typing inversion for $\Pi(x : U).T$ we get that T is typable with some sort. Then what is needed follows from substitution lemma.
- CONV: what is needed is present in the premise.

□

Totality is not a strictly necessary requirement — similar true propositions can be stated for non-total PTS, for example in CoC typing yields type which is either itself typable or equal to $\square$. But PTS-*rk* is indeed total so the theorem above is enough for our purposes.

Theorem 4 (Subject reduction). *In a total PTS, the following rule is admissible:*

$$\frac{\Gamma \vdash t : T \quad \vdash \Gamma \quad t \rightarrow_{\beta} t'}{\Gamma \vdash t' : T}$$

Proof. By structural induction over derivation of $t \rightarrow_\beta t'$. Inversion of a typing judgement yields all necessary info, albeit because of it types are now given only up to definitional equality so we have to apply conversion rule in each case. Crucially, it requires the type we convert into to be typable by a sort, so that's where we use Theorem 3.

The only tricky case are reductions of a domain type in $\Pi(x : T).U$ and $\lambda(x : T).t$. Let us consider the case of the following reduction:

$$\lambda(x : T).t \rightarrow_\beta \lambda(x : T').t.$$

In order to complete this case, we have to prove that $\Gamma, x : T' \vdash t : U$ given that $\Gamma, x : T \vdash t : U$. By appropriately strengthening the statement, we get that it is enough to prove admissibility of the following rule called *context reduction*:

$$\frac{\Gamma, x : U, \Delta \vdash t : T \quad \vdash \Gamma, x : U, \Delta \quad U \rightarrow_\beta U'}{\Gamma, x : U', \Delta \vdash t : T}$$

The proof is, again, by induction on the typing derivation. The only non-trivial case is with VAR rule used on x ; it is taken care of by appropriate use of CONV rule to convert $x : U'$ to $x : U$. To use conversion, we have to prove that U is well-typed; thankfully, it follows from $\vdash \Gamma, x : U, \Delta$.

Note that context reduction is admissible in arbitrary PTS! We don't need it anywhere else, though, so we didn't state it as a separate lemma. $\square$

Characterization of definitional equality, uniqueness of typing and subject reduction together imply the following corollary we're going to use a few times in our normalization proof:

Corollary 2 (Type stability). *In a total functional PTS, the following holds:*

$$\left\{ \begin{array}{l} \Gamma \vdash t : T \\ \Gamma \vdash t' : T' \\ \vdash \Gamma \\ t =_\beta t' \end{array} \right. \Rightarrow T =_\beta T'.$$

2.3 Properties of strongly normalizing terms

We're almost done with classical results! What is left now is to state which terms are strongly normalizing and collect properties of s.n. terms which allow our soundness proof to go through.

Informally, a term is strongly normalizing iff there are no infinite reduction chains starting from it. In constructive setting and in formalization, the following definition is a bit better:

Definition 5 (Strongly normalizing term). Set of s.n. terms is the smallest set sn in which the following rule is valid:

$$\frac{\forall t', (t \rightarrow_\beta t') \Rightarrow t' \in \text{sn}}{t \in \text{sn}}.$$

In other words, a term is strongly normalizing iff all possible reductions from it are strongly normalizing.

The following properties are then provable by definition of **sn** and by structural induction on available derivations of **sn** for terms in question:

Lemma 4 (Structural properties of **sn**).

$$x, s \in \mathbf{sn} \quad \frac{T \in \mathbf{sn} \quad U \in \mathbf{sn}}{\Pi(x : T).U \in \mathbf{sn}} \quad \frac{T \in \mathbf{sn} \quad t \in \mathbf{sn}}{\lambda(x : T).t \in \mathbf{sn}} \quad \frac{t u \in \mathbf{sn}}{t \in \mathbf{sn}} \quad \frac{t[u/x] \in \mathbf{sn}}{t \in \mathbf{sn}}$$

Application, of course, is the only nontrivial case. In case of β -reduction, strong normalization of both $(\lambda(x : T).t)$ and u doesn't imply anything about normalization behaviour of $t[u/x]$: consider a classical example from untyped lambda calculus, $\omega = (\lambda(x : T).xx)$ and $\Omega = \omega\omega$; while obviously not typable, ω is strongly normalizing, while Ω decidedly is not.

Situation changes when a head of application is not an abstraction at all. Thus we arrive at the definition of an *atomic term*.

Definition 6 (Atomic term). Set of atomic terms is the smallest set satisfying the following conditions:

1. $s, \Pi(x : S).T$ and y are atomic for all variables x, y , sort s and types S, T ;
2. If t is atomic, then tu is also atomic.

Atomic terms are useful precisely because their strong normalization survives application:

Lemma 5. *If $t, u \in \mathbf{sn}$ and t is atomic, then $tu \in \mathbf{sn}$.*

Proof. By induction on derivations of **sn** for t and u . Consider possible redexes in tu : tu is not itself a redex since t is atomic and hence not a lambda; the only possible redexes are either inside t or inside u . Applying corresponding induction hypotheses, we reach the desired result. The only caveat is that, for induction to go through, we need to check that atomicity of t is preserved by reduction; this is indeed true by induction on derivation of atomicity for t . $\square$

A dual view on strong normalization of application is the following: if we already know that $t[u/x] \in \mathbf{sn}$, then the most tricky case is already taken care of.

Lemma 6.

$$\left\{ \begin{array}{l} t[u/x]u_1 \dots u_n \in \mathbf{sn} \\ U, u \in \mathbf{sn} \end{array} \right. \Rightarrow (\lambda(x : U).t)uu_1 \dots u_n \in \mathbf{sn}.$$

Proof. By induction on n .

- $n = 0$. We need to prove that if $t[u/x], u, U \in \mathbf{sn}$, then $(\lambda(x : U).t)u \in \mathbf{sn}$. Note that $t[u/x] \in \mathbf{sn}$ implies $t \in \mathbf{sn}$; then, by structural induction on derivations of $\mathbf{sn}$ for t, u, U we get what we want by considering all possible reductions.
- $n \rightarrow (n + 1)$. By induction hypothesis, it is enough to show the following:

$$\left\{ \begin{array}{l} (\lambda(x : U).t)uu_1 \dots u_n \in \mathbf{sn} \\ t[u/x]u_1 \dots u_{n+1} \in \mathbf{sn} \end{array} \right. \Rightarrow (\lambda(x : U).t)uu_1 \dots u_n u_{n+1} \in \mathbf{sn}.$$

This is true by structural induction on available evidence.

□

3 Soundness of PTS- rk

3.1 Strong normalization

Theorem 5 (Strong normalization of PTS- rk).

$$\Gamma \vdash t : T \wedge \vdash \Gamma \Rightarrow t \in \mathbf{sn}.$$

Proof. By induction on the typing derivation. Case CONV is trivial; due to Lemma 4, cases AX, RULE, VAR and ABS present no problem as well. The only problematic case is application:

$$\left\{ \begin{array}{l} \Gamma \vdash u : \Pi(x : T).U \\ \Gamma \vdash t : T \\ \vdash \Gamma \\ t, u \in \mathbf{sn} \end{array} \right. \Rightarrow ut \in \mathbf{sn}$$

Now, here is the part where stratification according to ranks comes in useful. As typing yields correct type, there is some r such that $\Gamma \vdash \Pi(x : T).U : rk_r$. Let us explicitly add this to the list of premises; we will call the following proposition *Application Lemma*:

$$AL_r \triangleq \left\{ \begin{array}{l} \Gamma \vdash \Pi(x : T).U : rk_r \\ \Gamma \vdash u : \Pi(x : T).U \\ \Gamma \vdash t : T \\ \vdash \Gamma \\ t, u \in \mathbf{sn} \end{array} \right. \Rightarrow ut \in \mathbf{sn}$$

What indexing by r bought us is that now we can try proving this lemma by induction on r . Again, as it always is with strong normalization of application, the problem is in case where application head is a lambda and a β -reduction takes place. This calls for another lemma, *Substitution Lemma*:

$$SL_r \triangleq \left\{ \begin{array}{l} \Gamma, x : T, \Delta \vdash u : U \\ \Gamma \vdash T : rk_r \\ \Gamma \vdash t : T \\ \vdash \Gamma \\ t, u \in \mathbf{sn} \end{array} \right. \Rightarrow u[t/x] \in \mathbf{sn}$$

A proof of AL_r for all r (which is what we're after) is done by complete mutual induction with SL_r . Thus there are two main parts:

- $(\forall s < r. SL_s) \Rightarrow AL_r$: by induction on derivations of $u, t \in \text{sn}$. Consider possible reductions of ut : most of them happen either in u or in t , so that results of reductions from ut are strongly normalizing by inductive hypotheses. The only problematic case is when $u = \lambda(x : T').v$ and the reduction is β -reduction. By inversion of typing for u , it should be the case that $T' =_\beta T$; and by inversion of typing for $\Pi(x : T).U$, we get that $T : \text{rk}_s$ and $U : \text{rk}_t$ such that $r = \max(s + 1, t)$. Then, by type stability, $T' : \text{rk}_s$, and furthermore, $s < r$, so we can apply substitution lemma.
- $(\forall s \leq r. AL_s) \Rightarrow SL_r$. So we need to prove $u[t/x] \in \text{sn}$. We do it by simultaneous induction on derivation of $u \in \text{sn}$ and structural induction on u . As substitution propagates through u , most cases are satisfied by Lemma 4. A funny case is when $u = x$: then, a premise that $t \in \text{sn}$ helps. The only problematic case is application $u = u_1 u_2$; we consider three cases here:
 - $u_1[t/x]$ is atomic — handled by Lemma 4.
 - u_1 is not atomic — by virtue of it being well-typed, it means that $u = (\lambda(x : V).u')v_1 \dots v_k u_2$. All possible reductions from $u[t/x]$ are then handled by induction hypotheses.
 - u_1 is atomic, but $u_1[t/x]$ is not — by virtue of u_1 being well-typed, the only case when this is possible is when $u_1 = xv_1 \dots v_k$, thus $u[t/x] = t(v_1[t/x]) \dots (v_k[t/x])(u_2[t/x])$. From induction hypotheses we know that all $v_i[t/x]$ and $u_2[t/x]$ are s.n.; t is s.n. by premise and its rank is r . Then $tv_1[t/x]$ is s.n. by application lemma for r ; but note that rank of $tv_1[t/x]$ is not greater than r so application lemma applies again. Thus, by induction on k , the whole chain of applications is s.n.

□

This is a retelling of a proof that our friend, Aleksandr Levin, got after half-jokingly inputting the following prompt into ChatGPT:

Consider the following pure type system:

```
S = { rk_n | n natural }
Axiom = { (rk_n, rk_0) | n natural }
Rule = { (rk_m, rk_n, rk_max(m+1,n)) | m, n natural }
```

Is it strongly normalizing or not?

After approximately 20 minutes, LLM outputted the proof which we retold above. Looking at it in retrospect, it seems as if we stratified functions by ranks

with intent for such a direct proof to go through; honestly, we didn't think of that and were eagerly trying to adapt well-known approaches for proving normalization of pure type systems employing either saturated sets or candidates of reducibility, which didn't work because our "hierarchy" of ranks is not well-founded enough because of axioms $\text{rk}_n : \text{rk}_0$. This proof, however, doesn't even bother with it which means that we could pick *any* relation as Ax . Nevertheless, putting all sorts into rk_0 seems the most sensible approach to us since, informally, sort is kind of a constant which cannot be converted into a function.

3.2 Set-theoretical model

4 Formalizing normalization of PTS- rk in Rocq

Inductive characterization of the set of strongly normalizing terms follows [1].

5 Conclusion

References

- [1] Felix Joachimski and Ralph Matthes. "Short proofs of normalization for the simply-typed λ -calculus, permutative conversions and Gödel's T". In: *Archive for Mathematical Logic* 42.1 (2003), pp. 59–87. DOI: 10.1007/s00153-002-0156-9. URL: <https://doi.org/10.1007/s00153-002-0156-9>.